

skillgraph-mcp: Progressive Disclosure and Semantic Skill Graphs for Multi-Server MCP Deployments

Javier Rodeiro Rodríguez
github.com/jrodeiro5/skillgraph-mcp

May 2026

Abstract

As AI agents connect to increasing numbers of Model Context Protocol (MCP) servers, tool proliferation degrades agent performance. An agent wiring ten MCP servers may receive over 100 tool schemas in its context window before any user message is processed—a phenomenon we term *tool bloat*. We present **skillgraph-mcp**, an open-source MCP gateway built on top of skillful-mcp [6] that addresses tool bloat through two mechanisms: (1) *progressive disclosure*, where the agent sees a compact skill index upfront and retrieves per-server schemas on demand, and (2) a *semantic skill graph* encoding prerequisite and successor relationships between tools, enabling structured workflow planning. In a deployment of ten downstream servers exposing 113 tools, skillgraph-mcp reduces upfront context consumption from approximately 3,660 tokens to 287 tokens—a **92% reduction**—while preserving full tool access. The gateway becomes beneficial at four or more servers (approximately nine tools); below this threshold the gateway overhead exceeds the schema cost it replaces. We further describe **SkillOpt**, a background loop that uses execution traces to continuously refine tool descriptions and graph relations via LLM feedback without manual annotation. skillgraph-mcp is production-ready, ships prebuilt binaries for six platforms, and is available as open source [5].

1 Introduction

The Model Context Protocol [1] has rapidly become the standard integration layer between AI agents and external capabilities. An agent developer today may connect 10–20 MCP servers—search, memory, browser automation, code intelligence, databases—each exposing anywhere from 1 to 30+ tools.

This creates a structural problem: MCP clients load all registered server schemas at connection time. Every tool’s name, description, and JSON Schema `inputSchema` enters the model’s context window before the user types a single word. With ten servers and 113 tools, this pre-task overhead reaches approximately 3,660 tokens. At 20 servers, it approaches 7,000 tokens—a significant fraction of the effective working context, consumed by capability declarations rather than task-relevant content.

We observe that most agent tasks use a small subset of available tools. A user asking “search for papers on attention mechanisms” needs brave-search or firecrawl—not Playwright, gitnexus, wsl-terminal, or postgres. Yet all of their schemas are loaded regardless.

This paper presents skillgraph-mcp, a gateway that applies *progressive disclosure* to MCP tool loading. The agent sees a 10-line skill index upfront and requests per-server schemas only when needed. We report empirical measurements of the token savings, characterize when the gateway is and is not beneficial, describe the semantic skill graph that guides tool selection, and present SkillOpt—a trace-driven self-improvement loop that refines the graph without annotation.

2 Background and Related Work

Tool bloat in MCP deployments. Tool bloat was first described by Van Gent [7] in the context of skillful-mcp [6], the upstream project this work builds on. Van Gent observed that context consumption grows linearly with registered servers and proposed a gateway architecture with eight fixed discovery tools as the solution. skillgraph-mcp extends this with a semantic graph structure and the SkillOpt loop.

Enterprise MCP gateways. Microsoft MCP Gateway [3] addresses the operational concerns of multi-server deployments: Kubernetes lifecycle, RBAC, and adapter routing. These systems solve the *deployment* problem; they do not reduce agent context consumption. skillgraph-mcp solves the *cognition* problem and is complementary.

Client-side progressive disclosure. NousResearch’s Hermes tool-search [4] implements progressive disclosure at the *client* layer: three bridge tools replace all tool schemas, activating when tool context exceeds 10% of the context budget. skillgraph-mcp implements the same concept at the *server* layer, making it transparent to the client and compatible with any MCP client without modification.

Context budget management. Claude Code v2.1.105 introduced `skillListingBudgetFraction` [2], which limits the fraction of context reserved for skill descriptions. When exceeded, least-used skill descriptions are collapsed to bare names. This is a client-side graceful degradation mechanism operating on skill metadata; skillgraph-mcp provides server-side schema hiding that prevents schemas from entering the context at all. The two mechanisms are complementary.

3 System Design

3.1 Gateway Architecture

skillgraph-mcp acts as a single MCP server to the client and a multi-server client to N downstream MCP servers. It exposes exactly eight gateway tools regardless of how many downstream servers are connected (Table 1).

Tool	Purpose
<code>list_skills</code>	Compact index: skill name + one-line description
<code>use_skill</code>	Full tool schemas for one skill, on demand
<code>execute_code</code>	Python sandbox; all downstream tools callable by name
<code>plan_workflow</code>	Skill graph traversal for multi-step task planning
<code>get_skill_graph</code>	Full graph dump (nodes + edges)
<code>read_lattice</code>	Generated docs (<code>skills.md</code> , <code>relations.md</code>)
<code>read_resource</code>	Downstream MCP resources
<code>register_server</code>	Runtime server registration (loopback-only)

Table 1: The eight gateway tools exposed to the agent, independent of server count.

3.2 Progressive Disclosure Mechanics

At session start, the agent receives `list_skills` output: one line per skill with name and description (≈ 28 tokens/skill at 10 skills = 287 tokens total). To use a specific server, the agent calls `use_skill(skill_name)` and receives that server’s full tool schemas (≈ 32 tokens/tool empirically). Unused servers cost zero additional tokens beyond their single listing entry.

The crossover point—where gateway overhead equals raw schema cost—occurs at approximately four servers (≈ 9 tools). Below this threshold, deploying the gateway increases rather than decreases context consumption.

3.3 Semantic Skill Graph

An in-memory directed graph (3 node types: Skill, Tool, Resource; 5 edge types: HAS_TOOL, PREREQUISITE_FOR, PRODUCES, REQUIRES, COMMON_NEXT_STEP) encodes relationships between tools across servers. Example: `resolve_library_id` is PREREQUISITE_FOR `query_docs`. The agent calls `plan_workflow` to retrieve a traversal over this graph for a given task description, receiving an ordered list of tool calls to execute.

3.4 SkillOpt: Trace-Driven Self-Improvement

Two background goroutines run continuously:

Bootstrap loop. On startup, fetches GitHub READMEs for each downstream server, sends tools and README to an LLM, and generates initial descriptions and graph relations seeded into the configuration file.

Refinement loop. Polls trace files written by `execute_code` every 30 seconds. Batches up to 10 trajectory files (code, tool calls, results, errors) and asks the LLM to propose edits to tool descriptions and graph edges. Proposed edits are validated (hallucinated tool names are rejected) and merged atomically via temp-file rename, then `RebuildGraph` is called. The LLM provider is configurable; the loop skips silently if unconfigured.

4 Evaluation

4.1 Experimental Setup

We deploy skillgraph-mcp with ten downstream MCP servers spanning 113 tools (Table 2). All measurements taken on a local WSL2 environment (Ubuntu, Intel x86-64). Token estimates use 4 characters per token (conservative; JSON Schema inputs with deeply nested objects tokenize at a higher ratio). The per-tool token rate (32.4 tokens/tool) is empirically measured from the `use_skill("playwright")` response (23 tools, 2,981 characters, 745 tokens).

Server	Tools	Category
chrome-devtools	29	Browser automation
playwright	23	Browser automation
firecrawl	20	Web extraction
wsl-terminal	10	Shell / SSH
gitnexus	13	Code intelligence
memory	9	Agent memory
magic	4	UI generation
context7	2	Documentation
brave-search	2	Web search
sequential-thinking	1	Reasoning
Total	113	

Table 2: Ten downstream MCP servers in the evaluation deployment.

4.2 Upfront Token Cost

Table 3 and Figure 1 show the primary result: a 92% reduction in upfront tokens consumed at session start.

Configuration	Tools	Tokens
Raw (all schemas loaded)	113	$\approx 3,661$
skillgraph-mcp (<code>list_skills</code>)	10 skills	287
Reduction	91%	92%

Table 3: Upfront token cost: raw loading vs. skillgraph-mcp gateway.

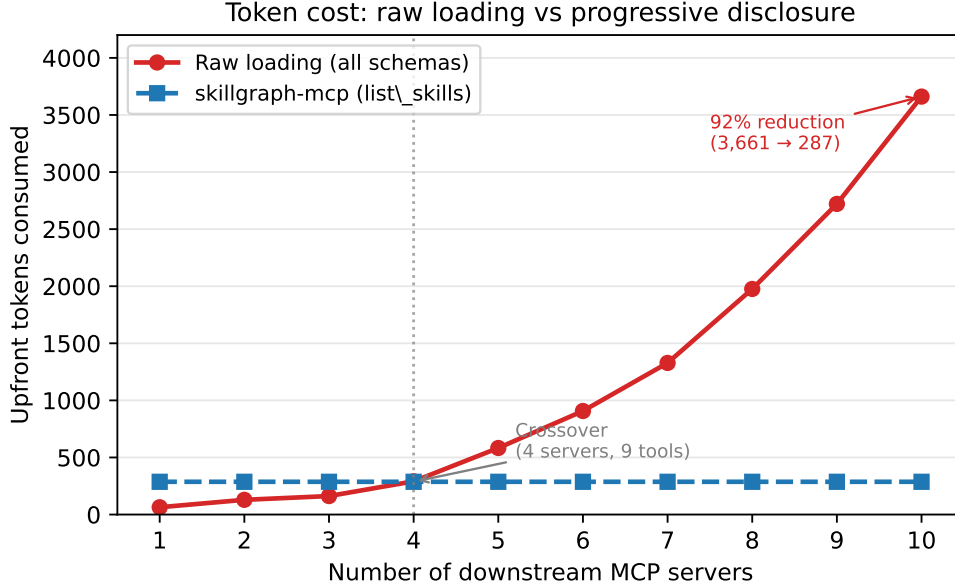


Figure 1: Token cost as downstream servers are added (smallest to largest by tool count). Gateway cost is constant at 287 tokens; raw cost grows linearly. Crossover at ≈ 4 servers / 9 tools.

4.3 Per-Task Cost Model

Figure 2 shows the per-task cost model. An agent using no `use_skill` calls (task solved via context alone) pays 287 tokens. An agent activating memory (9 tools, ≈ 288 tokens) pays 575 tokens total. An agent activating playwright (23 tools, 745 tokens) pays 1,032 tokens—versus 3,661 tokens for raw loading. The savings increase as more servers are registered; tasks that use only small servers benefit most.

4.4 Demo Interaction

The following condensed transcript captures an actual agent session through the gateway. The agent stores research findings in persistent memory, touching only one of ten registered servers:

Listing 1: Agent interaction via skillgraph-mcp gateway.

```
# Step 1: Agent calls list_skills (287 tokens, all 10 servers visible)
>> list_skills()
<< brave-search, chrome-devtools, context7, firecrawl, gitnexus,
    magic, memory, playwright, sequential-thinking, wsl-terminal
    (9 servers / 104 tools never load their schemas)

# Step 2: Agent inspects memory skill (288 tokens, 9 tools revealed)
>> use_skill("memory")
<< create_entities, add_observations, search_nodes,
    create_relations, read_graph, open_nodes,
```

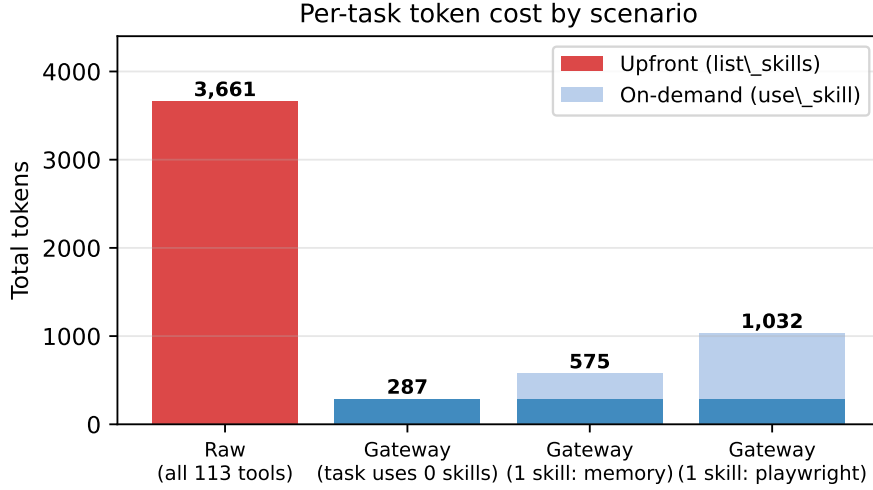


Figure 2: Total token cost per task scenario. “On-demand” bars show the additional cost of calling `use_skill` for one server.

```

delete_entities, delete_observations, delete_relations

# Step 3: Agent chains 3 tools in one execute_code call
>> execute_code("""
    create_entities(entities=[{
        "name": "skillgraph-mcp", "entityType": "Project",
        "observations": ["92% upfront token reduction", ...]
    }])
    create_relations(relations=[
        {"from": "skillgraph-mcp", "relationType": "extends",
         "to": "skillful-mcp"}
    ])
    graph = read_graph()
    return f"{len(graph['entities'])} entities stored"
""")
<< "3_entities, 3_relations_stored"

# Total: 287 + 288 + ~150 = 725 tokens (raw baseline: 3,661)

```

5 Limitations and Future Work

No task accuracy measurement. We measure token cost reduction, not whether agents make better tool selections. A controlled experiment comparing task success rates with and without the gateway on standard agent benchmarks is left for future work.

SkillOpt not evaluated. The claim that SkillOpt improves routing accuracy over time is a hypothesis; no before-and-after evaluation has been conducted.

Fixed per-tool token rate. The 32.4 tokens/tool rate is measured from a single server (Playwright). Servers with complex nested JSON Schemas may exhibit higher rates.

Crossover threshold. Users with only a few large servers benefit earlier than the 4-server threshold suggests. Users with 1–3 small servers should not deploy the gateway.

Future directions. Measuring task success rate on standard agent benchmarks; evaluating SkillOpt description quality via blind human evaluation; implicit discovery mode (transparent `use_skill` calls); integration with persistent trace indexing for semantic retrieval of historically successful tool chains.

6 Conclusion

skillgraph-mcp demonstrates that a lightweight gateway layer applying progressive disclosure to MCP tool schemas reduces agent upfront context consumption by 92% in a 10-server deployment, with linear improvement as server count grows. The semantic skill graph and SkillOpt loop provide structural guidance and annotation-free improvement without modifying the MCP protocol or downstream servers. The crossover threshold (≈ 4 servers, ≈ 9 tools) provides a clear decision criterion for practitioners evaluating whether to deploy the gateway.

Acknowledgements. This work builds on skillful-mcp by Kurtis Van Gent (MIT License). The progressive disclosure architecture is due to Van Gent; this paper contributes the semantic skill graph, SkillOpt loop, CLI diagnostics, and empirical token measurements.

References

- [1] Anthropic. Model context protocol specification. <https://modelcontextprotocol.io>, 2024.
- [2] Anthropic. Claude code settings reference. <https://code.claude.com/docs/en/settings.md>, 2026.
- [3] Microsoft. MCP gateway. <https://github.com/microsoft/mcp-gateway>, 2025.
- [4] NousResearch. Hermes tool search. <https://hermes-agent.nousresearch.com/docs/user-guide/features/tool-search>, 2025.
- [5] Javier Rodeiro Rodríguez. skillgraph-mcp. <https://github.com/jrodeiro5/skillgraph-mcp>, 2026.
- [6] Kurtis Van Gent. skillful-mcp. <https://github.com/kurtisvg/skillful-mcp>, 2026.
- [7] Kurtis Van Gent. Skills and MCP. <https://kvg.dev/posts/20260125-skills-and-mcp/>, 2026.